

Software Localization and Arabization

-Synthesis Paper

Timothy Ghanem
Birzeit University, Palestine
timothy.ghanim@gmail.com

Mustafa Jarrar
Birzeit University, Palestine
mjarrar@birzeit.edu

ABSTRACT

Region (or locale) specific text translation allows software vendors to reach diverse regions around the world. Information evolution is continuing to grow and the current tools and approaches are hardly keeping up with the demand. Manual UI translation is laborious and very inefficient way to translate UI text. Automatic translation does produce results with considerable quality; however, it requires manual review before publishing the translation results. In addition, automatic translation engines do not recognize the context or the region for which the translation is targeted which further contributes to the inaccuracy of the translation. In this paper, we present a novel approach that is capable to find the most probable translation of a single word from a source language into the target language that best matches a specific region or locale.

This is a synthesis paper reflecting student research during the Data Engineering course, master's in software engineering at Birzeit University in Palestine.

Reference: Timothy Ghanem, Mustafa Jarrar: Software Localization and Arabization. Technical Report. Birzeit University, Palestine. 2019

Keywords

Software localization, l10n, translation memory, machine translation engine.

1. Introduction and Motivation

In order to for software vendors to be able to reach diverse regions and people with different cultures than the country where the software was originally developed, vendors have to adapt their software to the different languages and locales and their software must be able to fully represent the original value the vendor intends to deliver as though it was developed in the local language of the user. The practice to adapting a software to specific culture (including language) is called localization (or “l10n” for short). Software localization is not all about UI text translation from the original language to the local language, localization involves the layout of the UI elements, dialog windows, text orientation...etc. All of these issues and more make up the process of localizing a software.

A vital step of the localization process is translating the UI text into local languages. With the ever-growing evolution of information and the speed at which data is expected to be translated and the quality required for today's software, the

current tools and approaches (e.g. translation memories, machine translations, crowdsourcing...etc.) are hardly keeping up, especially for areas like e-Commerce where products are added frequently to the system. For example, imagine an online retailer such as Amazon wanting to localize their products to better serve worldwide consumers. In order for Amazon to keep up with products that are added daily to their website, Amazon would then have to have specialized translation teams, probably located in different geographical locations, to oversee that the product names best describe the local culture (or region) when users from their assigned regions use the retail website. Those teams will have to be able to constantly translate new products as they are added to the system in addition to the millions of existing products that are currently registered in the system. Such a decision will be very impractical and expensive for Amazon to make and therefore they will be very reluctant to employ such teams. This is probably why Amazon is only available in Simplified Chinese, German, Brazilian, Portuguese, or Spanish in addition to English [19].

In the attempt to meet the text translation in the software localization, many approaches were developed to map (or translate) the original text (in the international language) to the local user language (include cultural differences).

(Muntes, 2012) reports on a typical software localization process. The process is typically built-into the process of the software development lifecycle. It starts with developers writing code and adding text inside the UI elements (or probably using a resource file to extract the text value at runtime). After that, the UI text is extracted from the source code (or resource files) using special text extraction tools which creates the input for the translation process. The input is then given to translation memories for filtering and then a machine translation engine is used to produce the final text translation. The translation output is post-edited manually by human translators before finally imported back into the source code or resource files. Both translation memories and machine translation engines are discussed next.

1.1 Translation Memories

A translation memory (or TM) is a storage of the original e-content, typically broken into manageable pieces each in the length of a sentence and its human translations in one big

archive. The idea is that, overtime, enormous collections of sentences and their corresponding translations are built up in the archive. Translators can propose alternative translations as either “exact match” or “fuzzy match” or can reuse the same translation. The result is increased productivity of the translator and consistency of translations. The TM grows incrementally because new translation units (or TUs) are added during the translation process. Although TMs have been reported to provide a 25% - 60% productivity gain for translations. In other places, the productivity gain has come to an end where translator productivity remained at the average of 2.400 words per day for years. In addition, TMs suffer from the issue of context-awareness since they operate at the sentence level; therefore, there is a great danger that a translator will focus a lot on translating individual sentences without awareness of the context where this sentence will be used. In addition to 1:1 equivalences that exists in the TMs, TMs can also include 1:n and n:1 relations between the sentences from the source language to sentences in the target language [4].

1.2 Machine Translation (MT) Engines

Machine Translation is the use of computer software to translate text from one language to another. There are three types of MT engines: rule-based, phrase-based [8], statistical and hybrid engines which combine both rule-based and statistical [7]. Another emerging type of MT engines is the neural machine translation (NMT) [8] which some people think is probably the new state-of-art trend for machine translation engines. There is no definitive definition for the quality of a machine translation engine because it depends on the several parameters such as: how the text is processed, how the engine is initialized, what is the type of the engine (statistical, rule-based...etc.) and the language pair that is being used [5, 9]. NLP Test suites have proven NMT to have a significant improvement when an online system turned from phrase-based MT to NMT [9]. Machine translation tools that are used in software localization had a significant contribution to the localization process; it was reported that a human translator’s throughput has been increased for up to 50% which consequently decreased the time to market for companies that are pursuing localization for their software [2].

Another approach for collecting region-specific (or localized text translations) for a software is crowdsourcing. Crowdsourcing for software localization was first proposed by (Hunt, 2013) to achieve software localization at a minimal cost. The basic concept behind crowdsourcing is giving the end user the capability to change the current translation of the software. The software includes a list of approved translations by the software vendor (called original translations). The original translations are created using an automated translation tools and are expected to produce inaccurate results. The user is given the ability to create a new translation based on an original translation. The new translation can then be saved to a file and uploaded to the

vendor’s server in order to be available to other users and even for other uses (i.e. in other applications).

Both manual and machine text translation suffer problems that limit their qualities, which requires further testing and evidence from linguistics experts to validate that the final translation of UI text correctly and accurately reflects the true meaning of the original text. This is sometimes a very costly process especially for complex and rich user interfaces.

To our knowledge, there is currently no publicly available text translation service that is capable to produce localized terminologies for specific regions and locales. Therefore, in this paper, we present a novel proposal that attempts to fill this gap by producing better and more accurate localized text translations that better reflect the original text meaning. The approach presents the concept of a localization memory, which is very similar to the concept of translation memory but yet contains more accurate translations of source texts. In addition, the localization memory includes context-related information such as the region where a translation applies. Our approach is expected to have a high degree of accuracy, thus significantly reducing the time needed to review the translated text in the UI.

2. Research Questions

The goal of this paper is to provide a novel approach to translate text between any language pair which is able to target a specific region or locale of the target language. More specifically, this paper intends to answer the following questions:

- What content is needed to extend the build bilingual dictionary?
- Given a word from a source language, how can it be mapped to another word in the target language that is most suitable for a particular region (or locale)?

3. Related Work

One of the primary challenges that accompany text translation for traditional software localization is the fact that the developers of the software and the localization experts are typically two separate teams that are (in large organizations) geographically separated. It is not expected from a software developer to be a linguistics expert nor is a translator expected to read source code. Therefore, there’s a gap when finding the proper translation for UI text when localizing a software [2]. Another consequence of this separation is an additional critical iterative step called Language Quality Assurance which is a highly resource and time-consuming process to collect the evidences and screenshots from the final product to make sure that the text translations are correct [2].

3.1 Translation Memories

There are several online translation memories available for users. MyMemory [10] claims to be the world’s largest translation memory. It’s freely available for users to use their

REST API to translate any text between any two language pairs. Also, it allows humans to contribute their own translations to the memory and currently includes over 4 billion human contributions. In addition to human contributions, MyMemory crawls the web for bilingual translations and stores them automatically to the archive. Linguee [11] is another online translation memory that contains over 1 billion translation, however its supported language pairs are mostly targeted towards European languages and missing Asian languages especially Arabic. Glosbe [12] is an online translation service which provides translations using multiple free dictionaries and a translation memory with over 1 billion sentences included.

The following table lists the closest localized translation each of the above tools were able to find for several product names. The translation is for English -> Arabic. Linguee [11] was excluded from the table because it does not include a translation from English -> Arabic. The localized Arabic strings are obtained from [15].

Word in English	Localization Synonyms	MyMemory	GlosBe
shoe	جزمة (مصر), سباط (الجزائر)	حذاء, نعل	حذاء, نعل
carpet	موكيت (الجزائر), زولية (عمان)	سجادة, بساط	سجادة, بساط
hammer	شاكوش (السعودية)	مطرقة	مطرقة, ثاكوث
towel	بشكير (الجزائر), توال (اليمن)	منشفة, فوطة	منشفة, بشكير
cake	جاثوه (مصر)	كيك, كعكة	كعكة

In general, translation memories do not provide context-specific information when mapping from the source sentence to the destination sentence. Therefore, we propose to extend TMs to include extra information about the mappings between the source language and the destination language such as the possible contexts (i.e. regions or locales) under which a mapping can exist. The result of including the context tag for the mappings will give a considerable hint to solve the context-awareness problem that was previously discussed. In addition, we propose to automatically assist existing TMs by the using the approach presented in this paper to automatically generate multiple versions of the same source sentence and mapping it to the same destination sentence by using the multiple synonyms that can be extracted for the same work. The same can be done by automatically generating multiple versions of the destination sentence and mapping it to the same source sentence. Assume that the TM includes a mapping 1:1 from

source language A to target language B and we have two equivalent synonyms sets S1 and S2. Then, using S1 we can substitute the words that are found in the source sentence with words from S2, thus generating a set of equivalent sentences that could be mapped to the target sentence. This way we transform 1:1 relation to n:1. In the same manner we can transform 1:1 to 1:n, n:1 to n:n, 1:n to n:n.

3.2 Crowdsourcing

Crowdsourcing is an excellent approach to localize a software. However, it suffers from several issues. Firstly, the user that is suggesting a different translation is not expected to be a linguistics expert which hinders the quality of the translation. Secondly, there is no guarantee that users will suggest a different translation. We propose that instead of using a base translation which might not be accurate and waiting for the user to submit a new translation, the system will detect the user's geolocation and based on that the system will consult the service model presented in the paper and from the localization tag the system can detect the translation that best suits the user's location. The system will also ask the user to confirm that the translation was accurate and even ask the user to submit a translation in the case the system was not able to find a localized translation in the localization memory.

3.3 Machine Translation (MT) Engines

Machine translation engines suffer from the problem of context-awareness when performing UI text translation. For example, machine translation engines do not differentiate if the same word appeared on a menu item or a button which might have different meanings. Even when context information is gathered from the UI, the current machine translation engines are not devised to exploit it. It has been observed that editing the translation that was produced by a machine translation engine to produce a publishable results is the most consuming part of a software localization process that utilizes a machine translation engine to translate the UI text [2].

We propose that MT engines be augmented with context-aware components and translation queries should include the sufficient contextual information for the engine to translate the input. Our method can be utilized to optimize MT engines by producing multiple translations for the same input using the different synonyms that were obtained. And, if contextual information is given as input to the translation engine, our localization function could be used to produce the most the accurate translation that best suits a specific region or locale. We also suggest that MT engines be augmented with crowdsourcing to further enhance the quality of the engine. The MT engine could suggest multiple translations for the same input for the same target locale, then human users can suggest to the system the best translation that matches the input.

There are several machine translation engines available online for users. In this document we will discuss three of the most famous engines: Google Translate, Microsoft Translator and DeepL. Google Translate is probably the most well-known online translation engine users will likely use to translate text and there are many other translation services that are built on top of Google Translate such as Standard Translator [16]. Google translate maps the source text into English and then to the target language [18]. It started as a statistical translation engine [17], it now has switched to using Google Neural Machine Translation [18]. The same story has been for Microsoft Translate which started as SMT and currently uses NMT. DeepL is claimed to be more advanced than Google Translate and Microsoft Translator; even though it supports 9 source languages and 8 target language, it can produce text translation that are more natural than what a human translator can produce [17].

Below is an attempt to run the same previous translations using Google Translate and Microsoft Translator. DeepL was not used because it doesn't support translating to Arabic.

Word in English	Localization Synonyms	Google Translate	Microsoft Translator
shoe	جزمة (مصر), سباط (الجزائر)	حذاء, نعل	حذاء
carpet	موكيت (الجزائر), زولية (عمان)	سجادة, موكيت	سجاد
hammer	شاكوش (السعودية)	مطرقة, شاكوش	مطرقة
towel	بشكير (الجزائر), توال (اليمن)	منشفة, فوطة	منشفة, فوطة
cake	جاتوه (مصر)	كعكة	كعكة

4. Our Approach

The approach that we present consists of three major parts: building a multilingual dictionary of words, generating equivalent synonym sets between every pair of the dictionary and then using a localization function that accepts a word from the source language, the target language and the region where the target language applies, the function would return the most probable equivalent word of the target language that best suits the provided region.

4.1 Building Multilingual Dictionary

Given multiple dictionaries between different pairs of languages, then the result dictionary is a single table that maps each word from any language to every other word in the other languages. More specifically, given the following source dictionaries {A, B}, {B, C}, {C, D}, then the resulting dictionary would look like the following, {A, B, C, D} for each word of A. Other sources for the

word mappings can be used such as DBpedia which provides multiple names, in different languages, for the same entity in the *label* predicate. Therefore, all the names for every word can be considered translations for the same word in different languages and can be added to the dictionary.

4.2 Building Synonym Sets

Using the approach presented in (Nasr, 2015), the equivalent synonym sets are extracted from the multilingual dictionary.

4.3 Localization Function

Using the different synonym sets that are extracted in the previous step, we have the necessary seed data to choose the best suitable synonym when translation between any language pair. The localization function is a statistical function that maintains a statistical database of the usage of each synonym in every region and locale. The localization function accepts a source word, target language and region (or locale) code, the function then returns the most probable synonym in the target language the best suits the specified region (or locale).

5. REFERENCES

- [1] <https://www.statmt.org/moses/index.php?n=Main.HomePage>
- [2] Muntés Mulero, V., Paladini Adell, P., España Bonet, C., & Márquez Villodre, L. (2012). Context-aware machine translation for software localization. In Proceedings of the 16th Annual Conference of the European Association for Machine Translation: EAMT 2012: Trento, Italy, May 28th-30th 2012(pp. 77-80).
- [3] Hunt, Tim. "Cost effective software internationalisation." Journal of Applied Computing and Information Technology 17.1 (2013): 2013.
- [4] Reinke, Uwe. "State of the art in translation memory technology." Language technologies for a multilingual Europe4 (2018): 55.
- [5] Garcia, Ignacio. "Beyond translation memory: Computers and the professional translator." The Journal of Specialised Translation 12.12 (2009): 199-214.
- [6] Nasr, Eman. Towards automatic construction of Arabic synonyms. Diss. Birzeit University, 2015.
- [7] https://kantanmt.com/documents/Machine_Translation.pdf.
- [8] Castilho, Sheila, et al. "Is neural machine translation the new state of the art?." The Prague Bulletin of Mathematical Linguistics 108.1 (2017): 109-120.
- [9] Burchardt, Aljoscha, et al. "A linguistic evaluation of rule-based, phrase-based, and neural MT engines." The Prague Bulletin of Mathematical Linguistics 108.1 (2017): 159-170.
- [10] <https://mymemory.translated.net/doc/>
- [11] <https://www.linguee.com/>
- [12] <https://glosbe.com/tmem/>
- [13] <https://www.ttmem.com/>
- [14] <https://omegat.org/>
- [15] <https://ar.mo3jam.com/>
- [16] <https://www.textunited.com/translate-files-now>
- [17] <https://www.textunited.com/blog/best-machine-translation-engines/>

- [18] https://en.wikipedia.org/wiki/Google_Translate
- [19] <https://www.amazon.com/gp/help/customer/display.html?ie=UTF8&nodeId=202085910>